14. SqLite DB

SQLite DB in Kotlin

Android Studio

Kotlin Android SQLite

SQLite is

- an **open-source**
- **relational database**
- used to perform **database operations** on **Android** devices, such as storing, manipulating or retrieving persistent data from the database.
- By default **SQLite** database is **embedded** in android. So, there is **no need to perform any database setup** or administration task.
- The **SQLiteOpenHelper** class **provides the functionality** to use the **SQLite** database.

SQLiteOpenHelper class

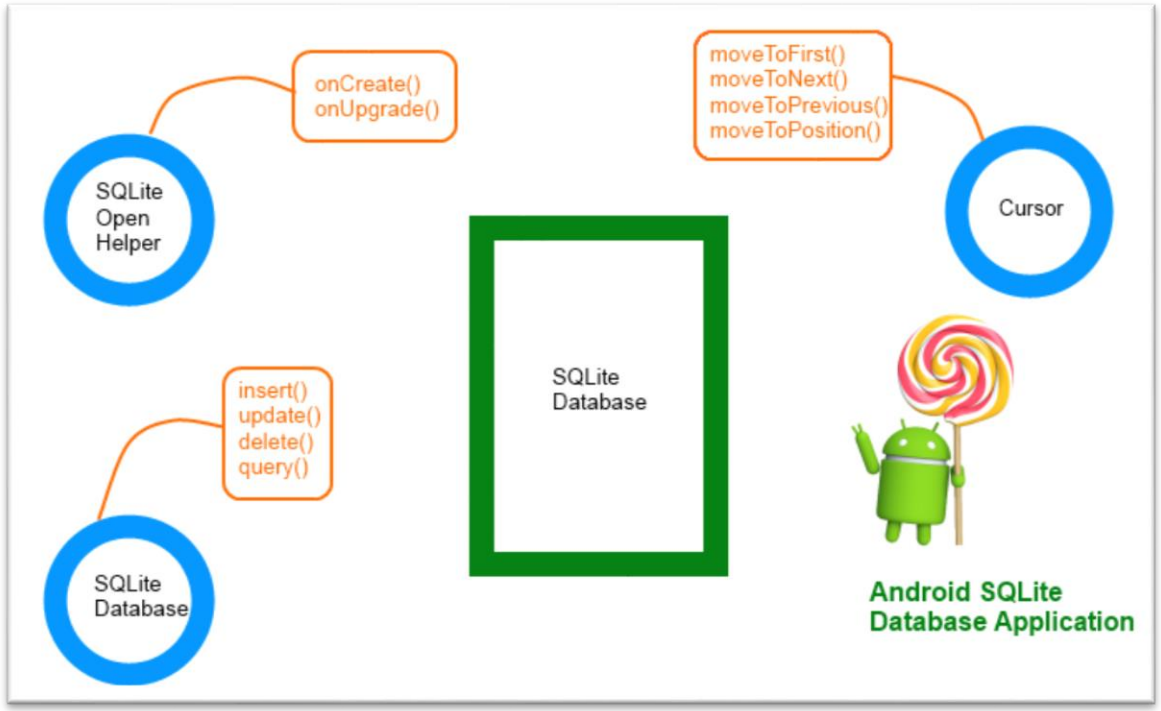
- The **android.database.sqlite.SQLiteOpenHelper** class is used for database creation and version management.
- For performing any database operation,
- you have to provide the implementation of **onCreate()** and **onUpgrade()** methods of SQLiteOpenHelper class.

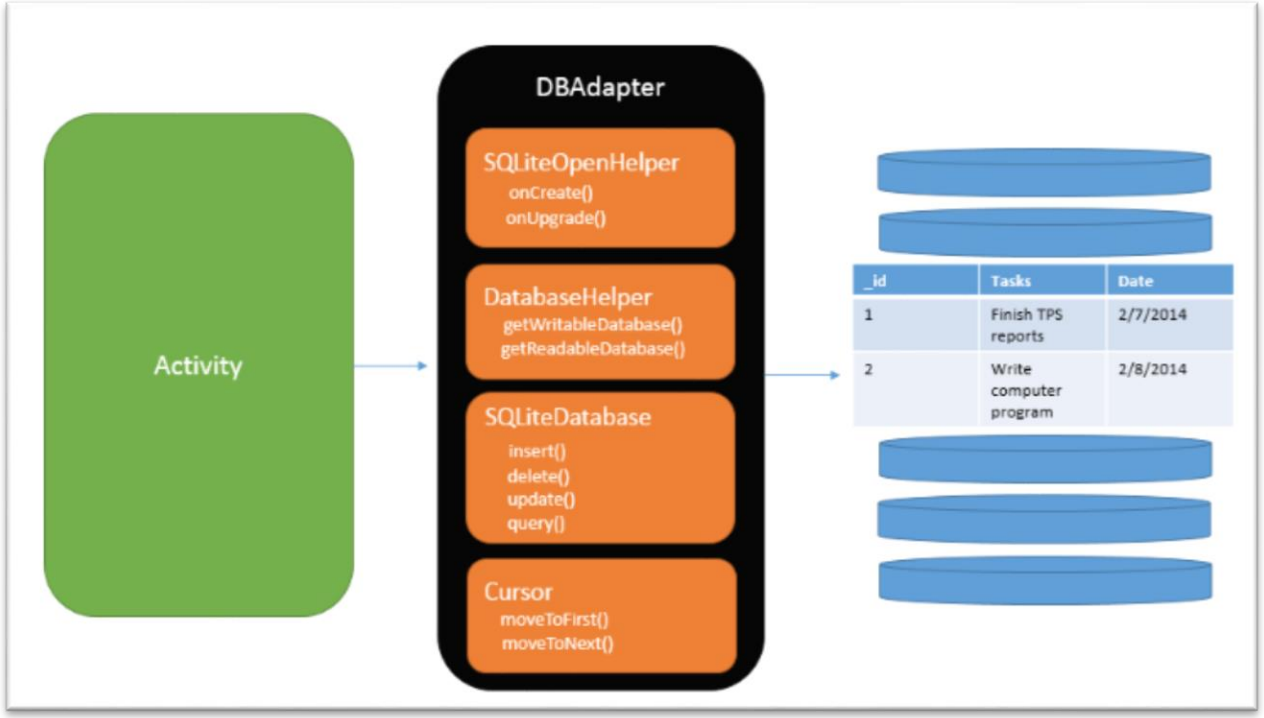
Kotlin Android SQLite

DBHelper class

- When implementing a database on Android,
- A common practice is to create a class which interacts
  - with the SQLite database and
  - also with the activities of the app.
- This class is **SQLiteOpenHelper** and contains all the necessary code
  - for creating the tables
  - and the assigned fields.
  - All of the operations regarding for inserting, updating, and deleting records.

Android SQLite Database



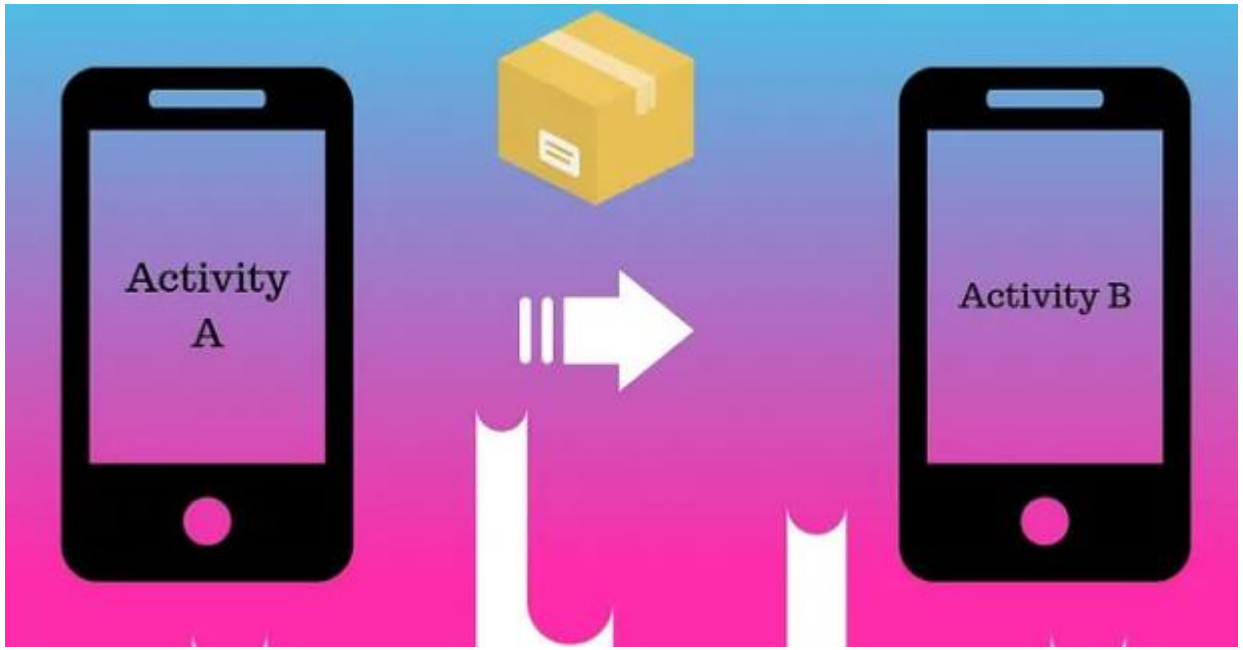


We can pass primitive data (Int, Float, Double, String etc.) by using Intent.

```
// send data
val intent = Intent(context, NextActivity::class.java)
intent.putExtra("name", “Value")
context.startActivity(intent)

// receive data
val name = intent.getStringExtra("name")
```

How to pass an object from one activity to another on Android in Kotlin

You can pass an object between activities in Android using **Parcelable** or **Serializable** interface.



1. Serializable

- Serializable is a markable interface or we can call as an empty interface. It doesn't have any pre-implemented method.
- Convert an object to byte stream.
- Serializable is standard **java** interface.
- Creation and passing data are very easy but it is slow process as compared to Parcelable.
- Serializable is a good choice to use when you want to store the state of an object to a file, a database or over a network.

Step 1-> make sure your object class implement the serializable

```
data class DummyObject(val name: String): Serializable
```

Step 2-> In the first activity, put the object in an Intent and start the second activity.

```
val intent = Intent(this, NextActivity::class.java)
val dummyObject = DummyObject(“Value")
intent.putExtra("object", dummyobject)
startActivity(intent)
```

Step 3 -> In Second Activity, get the object from the intent.

```
val object = intent.getSerializableExtra("object") as DummyObject
```

2. Parcelable

- More efficient and faster.
- Directly marshals and unmarshals the data to and from memory.
- Parclable is specific to Android and is the recommended method.

passing Cursor it not good idea

- Cursor is just a reference of your requites query you executed to SQLite,
- Passing Cursor it not good idea.
- Although if your data is not much sized then you can get data from Cursor and save it in any collection and pass it as serialized form to another activity via intent.
- Re-query your cursor on the destination side.
- Store the cursor data in a class which implements Parcelable.(when your cursor data is in small size).
- Above both method will work perfectly, concern really coming when you dealing with Static ones.
- *Remember: To open/close cursor and database when required.*

Android SQLite Database in Kotlin

Step By Step Implementation

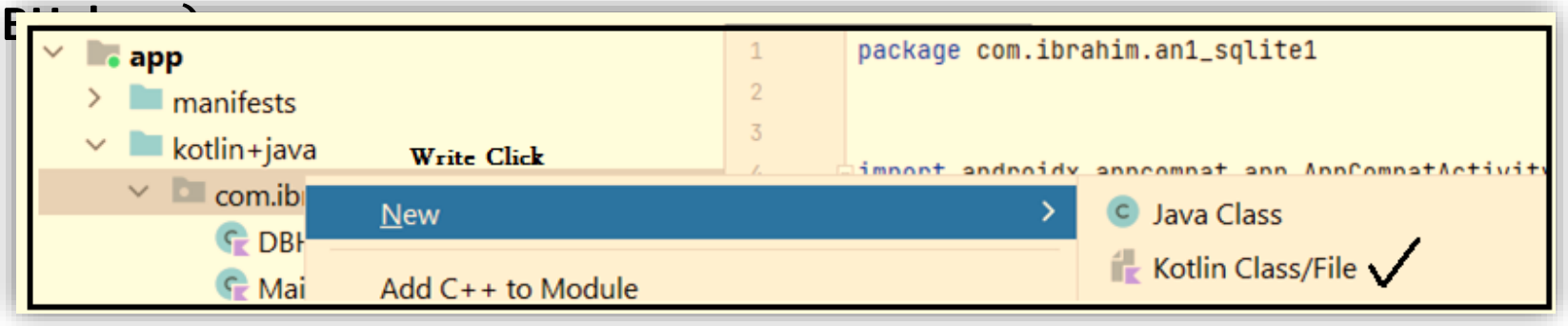
Step 1: Create a New Project (**AN1_SQLite1**).

Step 2: Giving permission to access the storage in the *AndroidManifest.xml* file

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Step 3: Working with the activity_main.xml file.

Step 4: Creating a new class for SQLite operations (**Kotlin class** and name it as **DB**



Step 5: Working with MainActivity.kt file

DBHelper class

```
1   package com.ibrahim.an1_sqlite1
2
3   import android.content.ContentValues
4   import android.content.Context
5   import android.database.Cursor
6   import android.database.sqlite.SQLiteDatabase
7   import android.database.sqlite.SQLiteOpenHelper
8
9   class DBHelper(context: Context, factory: SQLiteDatabase.CursorFactory?) :
10      SQLiteOpenHelper(context, DATABASE_NAME, factory, DATABASE_VERSION) {
11
12      // below is the method for creating a database by a sqlite query
13      override fun onCreate(db: SQLiteDatabase) {
14          // below is a sqlite query, where column names
15          // along with their data types is given
16          val query = ("CREATE TABLE " + TABLE_NAME + " ("
17                  + ID_COL + " INTEGER PRIMARY KEY, " +
18                  NAME_COl + " TEXT," +
19                  AGE_COL + " TEXT" + ")")
20
21          // we are calling sqlite
22          // method for executing our query
23          db.execSQL(query)
24      }
25
26      override fun onUpgrade(db: SQLiteDatabase, p1: Int, p2: Int) {
27          // this method is to check if table already exists
28          db.execSQL( sql: "DROP TABLE IF EXISTS " + TABLE_NAME)
29          onCreate(db)
30      }
```

```
31
32      // This method is for adding data in our database
33      fun addName(name : String, age : String ){
34
35          // below we are creating
36          // a content values variable
37          val values = ContentValues()
38
39          // we are inserting our values
40          // in the form of key-value pair
41          values.put(NAME_COl, name)
42          values.put(AGE_COL, age)
43
44          // here we are creating a
45          // writable variable of
46          // our database as we want to
47          // insert value in our database
48          val db = this.writableDatabase
49
50          // all values are inserted into database
51          db.insert(TABLE_NAME, nullColumnHack: null, values)
52
53          // at last we are
54          // closing our database
55          db.close()
56      }
```

DBHelper class

```
57
58      // below method is to get
59      // all data from our database
60      fun getName(): Cursor? {
61
62          // here we are creating a readable
63          // variable of our database
64          // as we want to read value from it
65          val db = this.readableDatabase
66
67          // below code returns a cursor to
68          // read data from the database
69          return db.rawQuery( sql: "SELECT * FROM " + TABLE_NAME,  selectionArgs: null)
70
71      }
72
```

```
72
73      companion object{
74          // here we have defined variables for our database
75
76          // below is variable for database name
77          private val DATABASE_NAME = "GEEKS_FOR_GEEKS"
78
79          // below is the variable for database version
80          private val DATABASE_VERSION = 1
81
82          // below is the variable for table name
83          val TABLE_NAME = "gfg_table"
84
85          // below is the variable for id column
86          val ID_COL = "id"
87
88          // below is the variable for name column
89          val NAME_COl = "name"
90
91          // below is the variable for age column
92          val AGE_COL = "age"
93      }
94  }
```

Main Activity

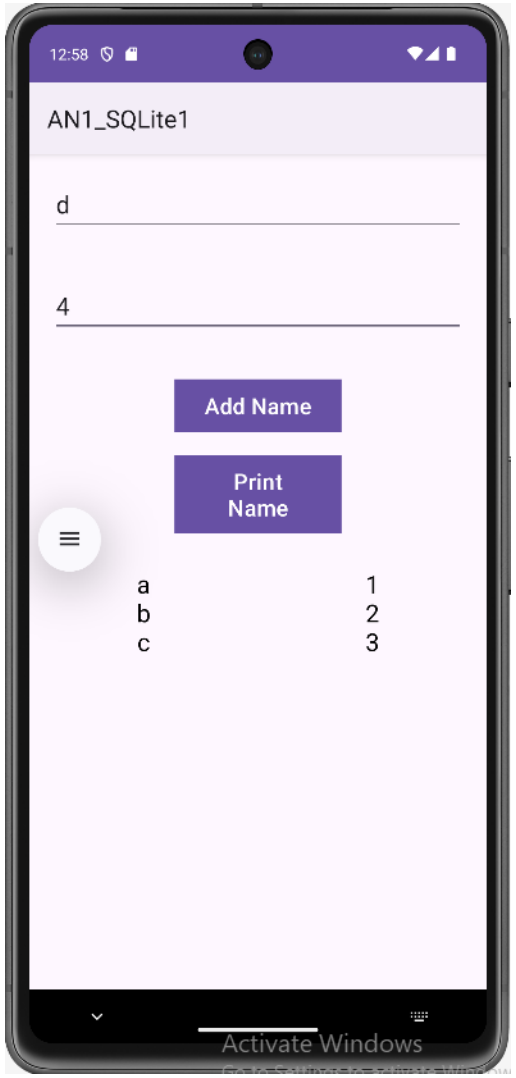
```
package com.ibrahim.an1_sqlite1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val enterName = findViewById<EditText>(R.id.enterName)
        val enterAge = findViewById<EditText>(R.id.enterAge)

        val addName = findViewById<Button>(R.id.addName)
        val printName = findViewById<Button>(R.id.printName)

        val Name = findViewById<TextView>(R.id.Name)
        val Age = findViewById<TextView>(R.id.Age)
```

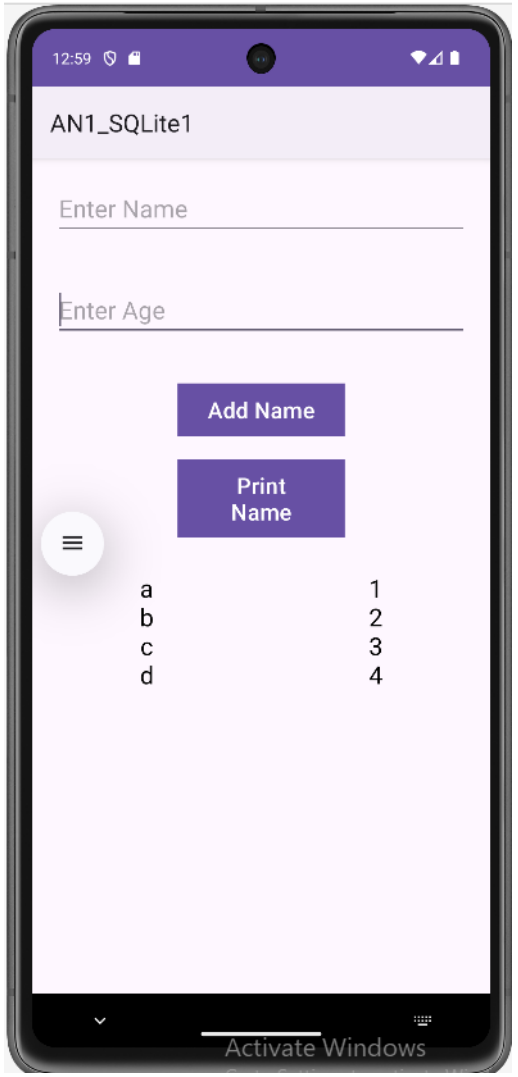
```
        // add data to table in database
        addName.setOnClickListener{ it: View!
            //  created  an object of DBHelper class
            val db = DBHelper( context: this,  factory: null)

            // get values from Name & Age
            val name = enterName.text.toString()
            val age = enterAge.text.toString()

            // pass data to database
            db.addName(name, age)

            // Toast to message on the screen
            Toast.makeText( context: this,  text: name + " added to database",
                    Toast.LENGTH_LONG).show()

            // clearing edittext
            enterName.text.clear()
            enterAge.text.clear()
        }
```

Main Activity

```
43          // retrieve data from database
44          printName.setOnClickListener{ it: View!
45              // clearing textview
46              Name.setText("")
47              Age.setText("")
48
49              //  created  an object of DBHelper class
50              val db = DBHelper( context: this,  factory: null)
51
52              // get all data from database
53              val cursor = db.getName()
54              // not null
55              if(cursor!!.moveToFirst() != null) {
56                  // moving the cursor to first position
57                  cursor!!.moveToFirst()
58                  //Name.append(cursor.getString(cursor.getColumnIndex(DBHelper.NAME_COl)) + "\n")
59                  Name.append(cursor.getString( columnIndex: 1) + "\n")
60                  Age.append(cursor.getString( columnIndex: 2) + "\n")
61
62                  // moving our cursor to next position
63                  while (cursor.moveToNext()) {
64                      // Name.setText(cursor.getString(1))
65                      Name.append(cursor.getString( columnIndex: 1) + "\n")
66                      Age.append(cursor.getString( columnIndex: 2) + "\n")
67                  }
68
69                  // at last we close our cursor
70                  cursor.close()
71              }
72          }
73      }
74  }
```

Output:


12:58
AN1_SQLite1
d
4
Add Name
Print Name
a
b
c
1
2
3
Activate Windows


12:59
AN1_SQLite1
Enter Name
Enter Age
Add Name
Print Name
a
b
c
d
1
2
3
4
Activate Windows